

ingenieurinformatik-buch-fcb5c.pages.gitlab.lrz.de/chapter/

2.6.1. Kontrollstrukturen (A) — Ingenieurinformatik 1 - Programmieren (Python)

Basiswissen

2. Informatik und Software Engineering

2.1. Computer

2.2. Die Von-Neumann-Architektur (S)

2.3. [Embedded?]

Software (S)

2.4. Codierung und Zahlensysteme

2.4.1. Beispiel Barcode (S)

2.4.2. Zahlen (S)

2.4.3. Zahlensysteme (S)

2.4.4. Zahlensysteme (A*)

2.5. Programmieren

2.5.1. Algorithmus (S)

2.5.2. Komplexität (S)

2.5.3. Software vs. Programm (S)

2.5.4. Repositories und Hubs (V*)

2.6. Programmstruktur

2.6.1. Kontrollstrukturen (A)

2.6.2. Datenstrukturen (A)

2.6.3. Übungsaufgabe: BMI (A*)

2.6.1.1. Fallunterscheidungen (bedingte Ausführung)

Eine Fallunterscheidung führt Code nur dann aus, wenn eine Bedingung erfüllt ist. Die Bedingung wird zur Laufzeit zu **True** oder **False** ausgewertet.

```
stamm_laenge = 20 # 20 m
if stamm_laenge < 15:
    print("Zu kurz. Wird anderweitig verwertet.")
else:
    print("Passt. Ab damit in die Produktion.")
```

Passt. Ab damit in die Produktion.

2.6.1.2. Wiederholung

Wiederholung bedeutet: Wir schreiben einen Ablauf einmal und führen ihn mehrfach aus. So können Programme abhängig von der Eingabe unterschiedlich viele Schritte machen (z. B. „solange, bis ...“).

Zwei zentrale Formen sind:

1. Iteration (Schleifen): Wiederholung über **for** oder **while**.
 - bestimmt: Die Anzahl der Durchläufe ist vorab bekannt (typisch: **for**).
 - unbestimmt: Die Anzahl hängt von einer Bedingung ab (typisch: **while**).
2. Rekursion: Wiederholung durch Selbstbezug.

Hinweis: Rekursion

Rekursion bedeutet grob: Man wendet „denselben Trick“ immer wieder auf kleinere Teile an, bis es nicht mehr weitergeht.

Beispiel (Biber und Stamm):

2.4.4. Zahlensysteme (A*)	58
2.5. Programmieren	70
2.5.1. Algorithmus (S)	70
2.5.2. Komplexität (S)	72
2.5.3. Software vs. Programm (S)	77
2.5.4. Repositories und Hubs (V*)	79
2.6. Programmstruktur	81
2.6.1. Kontrollstrukturen (A)	82
2.6.2. Datenstrukturen (A)	88
2.6.3. Übungsaufgabe: BMI (A*)	94
2.7. Kommunikation von Software	95
2.7.1. Nass-Shneiderman-Diagramme (A)	96
2.7.2. Pseudocode (V)	97
2.7.3. Vergleich Pseudocode - Struktogramm (V)	98
2.8. Takeaways	100
2.9. Übungsaufgabe: ICAO-Standardatmosphäre (A*)	101
III Python verstehen	103
3. Sprach Eigenschaften	105
3.1. Indentation (S)	106
3.2. Performance (S)	108
3.3. Varianten und Versionen (S)	111
3.4. Paradigmen (V)	114
3.5. Datenmodell	116
3.5.1. Typing (S)	119
3.5.2. Seiteneffekt (S)	120
3.5.3. Speicherverwaltung (V)	123
3.5.4. Zahlen (V)	123
3.6. Takeaways	125
IV Python anwenden	127
4. Übersicht	129
4.1. Was umfasst dieser Teil?	129
4.2. Wie arbeiten Sie mit diesem Teil?	130
5. Das Python-Ökosystem	131
5.1. Installation (Kurs-Setup)	132
5.1.1. Interaktive Website und JupyterHub (V)	132
5.1.2. Python-Umgebungen: Distribution, Umgebungen, Pakete (S)	132
5.1.3. Praktikum-Setup (V)	135
5.2. Python aufrufen	137
5.2.1. REPL (V)	138
5.2.2. Jupyter Notebook (V)	139
5.2.3. Script / Datei (V)	140
5.3. Python-Skripte: Dateien strukturieren	140
5.3.1. Python-Projekte (S)	141
5.3.2. Einfaches Pythonprojekt (S)	141
5.3.3. Praktikum-Python-Dateien (V)	144
5.4. Module und Pakete	145
5.4.1. Ein eigenes Modul (V)	145
5.4.2. Python's Modulpriorisierung (S)	146